

THEORETICAL REPORT

MODULE 3 – PYTHON BDD & RESTFUL AUTOMATIONS

Name: Saheli Deb

Enrollment No.: 12023052019071

Course: Python Automation

Report Type: Section-wise Theoretical Report

Prepared according to the prescribed Module 3 section structure.

3.1 GET Request and Status Validation

A GET request is used to retrieve a representation of a resource from a REST service. In Python automation, the Requests library provides a simple interface for sending HTTP requests. A test should not only check that a request was sent successfully; it should validate the HTTP status code and, where appropriate, important response fields.

Key Theoretical Concepts

- **GET method:** The client asks the server to return a resource. GET is normally used for retrieval and should not be used to create or modify server state.
- **URL and endpoint:** The endpoint identifies the resource being requested. Query parameters can be appended to filter, sort or otherwise control the response.
- **Status validation:** Common successful responses include 200 OK. Tests should explicitly assert the status expected for the operation.
- **Response body validation:** A successful status alone may not prove that the correct data was returned. JSON fields, identifiers and business values should also be checked.
- **Timeouts:** Automation should use a timeout so a test does not wait indefinitely for an unavailable service.

Illustrative Example

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10
)

assert response.status_code == 200
data = response.json()
assert data["id"] == 1
```

3.2 POST Request with JSON Payload and Headers

POST is commonly used to submit data to a REST service, often to create a new resource. In Python, a JSON payload can be supplied through the json argument of requests.post(). HTTP headers communicate metadata such as content type, accepted response format, authorization information and client identity.

Key Theoretical Concepts

- **Request body:** The body contains the data sent to the server. JSON is a common representation for REST APIs.
- **Content negotiation:** Accept indicates the response format the client can process; Content-Type describes the format of the request body.
- **Response code:** A successful creation commonly returns 201 Created, although the exact contract depends on the API.
- **Validation:** Tests should validate both the status code and important fields returned by the service.
- **Authentication header:** Protected APIs may use an Authorization header such as Bearer authentication.

Illustrative Example

```
import requests

headers = {
    "Accept": "application/json",
    "Content-Type": "application/json"
}

payload = {
    "title": "API Test",
    "body": "REST automation",
    "userId": 1
}

response = requests.post(
    "https://jsonplaceholder.typicode.com/posts",
    json=payload,
    headers=headers,
    timeout=10
)

assert response.status_code == 201
assert response.json()["title"] == "API Test"
```

3.3 Session and Authentication Pattern

A `requests.Session` object can persist common headers, cookies and connection-related state across multiple HTTP requests. This is useful when several API operations belong to one logical test flow. Authentication can be implemented using API keys, bearer tokens, basic authentication, cookies or another mechanism defined by the API contract.

Key Theoretical Concepts

- **Session object:** A Session provides a reusable client context instead of constructing every request independently.
- **Persistent headers:** Headers common to all requests can be configured once on the session.
- **Cookies:** A session can retain cookies returned by earlier requests, which is useful for cookie-based authentication.
- **Bearer token:** A token is commonly supplied through the Authorization header. Secrets should be stored outside source code, for example in environment variables.
- **Security practice:** Real credentials, access tokens and passwords should never be hard-coded into a lab repository.

Illustrative Example

```
import os
import requests

session = requests.Session()
session.headers.update({
    "Accept": "application/json"
})

token = os.getenv("API_TOKEN")

if token:
    session.headers["Authorization"] = f"Bearer {token}"

response = session.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10
)

assert response.status_code == 200
```

3.4 PUT, PATCH and DELETE

REST APIs use different HTTP methods to represent different operations. PUT is generally associated with replacing or updating a resource representation, PATCH with a partial modification, and DELETE with removing a resource. Automation should validate the status code and the response according to the documented API contract.

Key Theoretical Concepts

- **PUT:** Typically sends a complete representation of the resource or requests replacement of the current representation.
- **PATCH:** Typically carries only the fields that need to change.
- **DELETE:** Requests removal of the identified resource. APIs may return 200, 202 or 204 depending on their design.
- **Idempotency:** PUT and DELETE are generally defined as idempotent methods, meaning repeating the same operation has the same intended effect; PATCH behavior depends on its implementation.
- **Validation:** The test should confirm the documented status and, where possible, verify the resulting resource state.

Illustrative Example

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

put = requests.put(
    url,
    json={"id": 1, "title": "Updated", "body": "New body", "userId": 1},
    timeout=10
)
assert put.status_code == 200

patch = requests.patch(
    url,
    json={"title": "Patched"},
    timeout=10
)
assert patch.status_code == 200

delete = requests.delete(url, timeout=10)
assert delete.status_code in (200, 204)
```

3.5 Response Handling and Error Management

Reliable API automation must distinguish between successful responses, client errors, server errors and transport-level failures. The response object exposes the status code, headers, body and convenience methods such as `json()` and `raise_for_status()`. Error handling should produce useful diagnostics while allowing the test framework to record the failure correctly.

Key Theoretical Concepts

- **2xx responses:** Generally indicate that the request was successfully processed.
- **4xx responses:** Usually indicate a problem with the request or client-side conditions, such as 400 Bad Request, 401 Unauthorized, 403 Forbidden or 404 Not Found.
- **5xx responses:** Indicate that the server encountered an error while processing the request.
- **JSON parsing:** `response.json()` should only be used when the response body is expected to be JSON; malformed or non-JSON content should be handled appropriately.
- **raise_for_status:** This Requests method raises an `HTTPError` for unsuccessful 4xx/5xx responses and can simplify error propagation.
- **Network failures:** Connection errors, timeouts and DNS problems are not the same as HTTP status failures and should be handled separately when necessary.

Illustrative Example

```
import requests

try:
    response = requests.get(
        "https://jsonplaceholder.typicode.com/posts/999999",
        timeout=10
    )

    if response.status_code == 404:
        print("Resource not found")
    else:
        response.raise_for_status()

except requests.exceptions.Timeout:
    print("Request timed out")

except requests.exceptions.RequestException as exc:
    print("Request failed:", exc)
```

3.6 Behave BDD API Scenario

Behavior Driven Development (BDD) expresses software behavior in a business-readable form. Behave is a Python BDD framework that maps Gherkin statements to Python step definitions. For API automation, a feature can describe the expected behavior of an endpoint while the step implementation performs the actual HTTP request and validation.

Key Theoretical Concepts

- **Feature:** Describes a high-level capability or behavior being tested.
- **Scenario:** Represents one concrete example of that behavior.
- **Given:** Defines the initial context or precondition.
- **When:** Describes the action performed by the client.
- **Then:** Describes the expected observable result.
- **Scenario context:** Behave's context object can carry response data from one step to another.

Illustrative Example

Feature: Posts API

```
Scenario: Retrieve an existing post
    Given the posts API is available
    When I request post 1
    Then the response status should be 200
    And the response should contain post id 1
```

3.7 Scenario Outline, Hooks and Tags

Behave provides additional mechanisms for scaling BDD suites. A Scenario Outline allows one scenario template to execute with multiple sets of Examples data. Hooks such as `before_scenario` and `after_scenario` can prepare and clean up test resources. Tags allow scenarios or features to be grouped and selectively executed.

Key Theoretical Concepts

- **Scenario Outline:** Uses placeholders such as `{id}` and an Examples table to run the same behavior with multiple values.
- **Examples:** Contains the input data used for each Scenario Outline iteration.
- **Hooks:** `before_all`, `before_feature`, `before_scenario`, `after_scenario` and related hooks can perform lifecycle actions.
- **Tags:** Labels such as `@smoke` or `@api` allow selective execution and organization.
- **Maintainability:** BDD suites should keep feature files readable while placing implementation logic in step definitions and reusable support code.

Illustrative Example

Feature: Posts API

@smoke

Scenario Outline: Retrieve different posts

Given the posts API is available

When I request post <id>

Then the response status should be 200

Examples:

id
1
2
3

Example hook:

def before_scenario(context, scenario):

context.response = None

Run tagged scenarios:

behave --tags=@smoke

Module 3 – Overall Theoretical Summary

Module 3 connects Python programming with REST API automation and Behavior Driven Development. The progression begins with individual HTTP operations and response validation, moves through sessions and authentication, covers update and delete methods and systematic error handling, and then introduces BDD through Behave. Scenario Outline, Hooks and Tags provide mechanisms for data-driven execution, lifecycle management and selective test execution. Together, these concepts form a structured foundation for maintainable REST automation suites.

Module 3 Section Checklist

Section	Coverage
3.1	GET Request and Status Validation
3.2	POST Request with JSON Payload and Headers
3.3	Session and Authentication Pattern
3.4	PUT, PATCH and DELETE
3.5	Response Handling and Error Management
3.6	Behave BDD API Scenario
3.7	Scenario Outline, Hooks and Tags